

LAPORAN PRAKTIKUM
STRUKTUR DATA
“IMPLEMENTASI BINARY TREE DALAM BAHASA PEMROGRAMAN
JAVA”

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi Binary Tree dalam Bahasa Pemrograman Java. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 12 Juni 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan.....	4
1.3 Manfaat.....	5
BAB II PEMBAHASAN.....	6
2.1 Implementasi Node.....	6
2.1.1 Deklarasi atribut.....	6
2.1.2 Constructor Node	6
2.1.3 Method setter.....	7
2.1.4 Method getter	7
2.1.5 Method Traversal Preorder.....	8
2.1.6 Method Traversal Postorder	8
2.1.7 Method Traversal Inorder.....	9
2.1.8 Method Menampilkan Struktur Tree	9
2.2 Implementasi Binary Tree	10
2.2.1 Deklarasi Atribut	10
2.2.2 Construtctor BTree.....	10
2.2.4 Method Traversal Tree	12
2.2.5 Method Mengambil Root	12
2.2.6 Method Pemeriksaan Tree Kosong	13
2.2.7 Method Menghitung Jumlah Node.....	13
2.2.8 Method Menampilkan Struktur Tree	14
2.2.9 Method Getter dan Setter Current Node.....	14
2.2.10 Method Setter Root.....	14
2.3 Implementasi Binary Tree Driver	15
2.3.1 Method membuat Tree	15
2.3.2 Method Menambahkan Simpul Data.....	15
2.3.3 Menjadikan simpul 1 sebagai Root	16
2.3.4 Method Set root.....	16
2.4 Implementasi Graph Traversal.....	18

2.4.1	Deklarasi Atribut Graph	18
2.4.2	Menambahkan Edge pada Graf.....	18
2.4.3	Menampilkan Graph Awal.....	19
2.4.4	Traversal menggunakan DFS (Depth-First Search)	19
2.4.5	Traversal menggunakan BFS (Breadth-First Search).....	20
2.4.6	Implementasi dan Pengujian Graf.....	21
2.5	Latihan Membuat Peta Bandara dan Pencarian Interaktif menggunakan BFS dan DFS dengan menggunakan GUI	22
BAB III KESIMPULAN.....		33
3.1	Kesimpulan.....	33
DAFTAR PUSTAKA		34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pengembangan perangkat lunak karena digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara lebih efisien. Pemilihan struktur data yang tepat akan memengaruhi performa program, kemudahan pengelolaan data, serta efektivitas proses pencarian dan penelusuran.

Pada praktikum ini dilakukan implementasi beberapa konsep struktur data, yaitu Binary Tree dan Graph beserta algoritma traversal yang digunakan pada masing-masing struktur. Binary Tree digunakan untuk memahami hubungan antar simpul dalam bentuk hierarki serta proses penelusuran menggunakan metode preorder, inorder, dan postorder. Selain itu, dilakukan implementasi graf dengan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian jalur antar node.

Implementasi juga dikembangkan dalam bentuk antarmuka grafis menggunakan Java Swing agar proses traversal dapat diamati secara visual. Melalui visualisasi tersebut, perbedaan pola eksplorasi antara BFS dan DFS dapat diamati secara lebih jelas berdasarkan jalur yang dilalui dan jumlah node yang dieksplorasi.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dan implementasi struktur data Binary Tree.
2. Memahami proses traversal pada Binary Tree menggunakan metode preorder, inorder, dan postorder.
3. Memahami representasi graf menggunakan adjacency list.
4. Mengimplementasikan algoritma Breadth First Search (BFS) dan Depth First Search (DFS).
5. Membandingkan hasil traversal yang dihasilkan oleh BFS dan DFS.

6. Membuat visualisasi proses pencarian jalur menggunakan Java Swing.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menambah pemahaman mengenai penerapan struktur data pada program.
2. Melatih kemampuan dalam membangun hubungan antar node pada tree dan graf.
3. Memahami cara kerja algoritma traversal dalam proses pencarian data dan jalur.
4. Memberikan pengalaman dalam membuat visualisasi algoritma menggunakan antarmuka grafis.

BAB II

PEMBAHASAN

2.1 Implementasi Node

Class **Node_2511531002** digunakan untuk merepresentasikan satu simpul (node) pada struktur data binary tree. Setiap node menyimpan sebuah nilai bertipe integer serta memiliki referensi menuju anak kiri (left child) dan anak kanan (right child). Selain itu, class ini juga menyediakan method untuk mengatur data, mengambil data, melakukan traversal, serta menampilkan struktur tree.

2.1.1 Deklarasi atribut

```
1 package pekan9_2511531002;
2
3 public class Node_2511531002 {
4     int data_1002;
5     Node_2511531002 left_1002;
6     Node_2511531002 right_1002;
```

Kode Program 2.1 – Blok Program Deklarasi atribut

Bagian ini digunakan untuk mendefinisikan atribut yang dimiliki oleh setiap node. Variabel **data_1002** berfungsi menyimpan nilai utama dari node. Sementara itu, **left_1002** dan **right_1002** digunakan untuk menyimpan referensi menuju node anak kiri dan anak kanan. Nilai awal kedua atribut tersebut dapat berupa null apabila node belum memiliki cabang.

2.1.2 Constructor Node

```
7 public Node_2511531002(int data_1002) {
8     this.data_1002 = data_1002;
9     left_1002 = null;
10    right_1002 = null;
11 }
```

Kode Program 2.2 – Constructor Node

Constructor digunakan untuk membuat objek node baru sekaligus memberikan nilai awal pada node tersebut. Parameter **data_1002** akan disimpan ke atribut **data_1002**, sedangkan **left_1002** dan **right_1002** diinisialisasi dengan null yang menandakan node belum memiliki anak.

2.1.3 Method setter

```
12 public void setLeft_1002(Node_2511531002 node_1002) {  
13     if (left_1002 == null)  
14         left_1002 = node_1002;  
15 }  
16 public void setRight_1002(Node_2511531002 node_1002) {  
17     if (right_1002 == null)  
18         right_1002 = node_1002;  
19 }  
20 public void setData_1002(int data_1002) {  
21     this.data_1002 = data_1002;  
22 }
```

Kode Program 2.3 – Blok Program Method setter

Method setter digunakan untuk mengubah atau menambahkan nilai pada atribut node. Method **setLeft_1002()** digunakan untuk menambahkan anak kiri apabila posisi tersebut masih kosong. Method **setRight_1002()** bekerja dengan cara yang sama untuk anak kanan. Penggunaan kondisi if memastikan node yang sudah terisi tidak langsung tertimpa oleh node baru. Sedangkan **setData_1002()** digunakan untuk mengganti nilai data yang tersimpan pada node.

2.1.4 Method getter

```
23 public Node_2511531002 getLeft_1002() {  
24     return left_1002;  
25 }  
26 public Node_2511531002 getRight_1002() {  
27     return right_1002;  
28 }  
29 public int getData_1002() {  
30     return data_1002;  
31 }
```

Kode Program 2.4 – Blok Program Method getter

Method getter digunakan untuk mengambil atau mengakses nilai atribut dari luar class. Method **getLeft_1002()** mengembalikan node anak kiri, **getRight_1002()** mengembalikan node anak kanan, sedangkan **getData_1002()** digunakan untuk mengambil nilai data yang tersimpan pada node

2.1.5 Method Traversal Preorder

```
33 void printPreorder_1002(Node_2511531002 node_1002) {  
34     if (node_1002 == null)  
35         return;  
36     System.out.print(node_1002.data_1002 + " ");  
37     printPreorder_1002(node_1002.left_1002);  
38     printPreorder_1002(node_1002.right_1002);  
39 }
```

Kode Program 2.5 – Blok Program Method Traversal Preorder

Method ini digunakan untuk melakukan traversal tree dengan metode **Preorder**. Urutan kunjungannya adalah: **Root - Subtree kiri - Subtree kanan**. Method akan mencetak data node terlebih dahulu, kemudian memanggil dirinya sendiri secara rekursif ke bagian kiri dan kanan hingga seluruh node selesai dikunjungi.

2.1.6 Method Traversal Postorder

```
40 void printPostorder_1002(Node_2511531002 node_1002) {  
41     if (node_1002 == null)  
42         return;  
43     printPostorder_1002(node_1002.left_1002);  
44     printPostorder_1002(node_1002.right_1002);  
45     System.out.print(node_1002.data_1002 + " ");  
46 }
```

Kode Program 2.6 – Blok Program Method Traversal Postorder

Method ini digunakan untuk melakukan traversal tree dengan metode **Postorder**. Urutan kunjungannya adalah: **Subtree kiri - Subtree kanan - Root**. Traversal ini umum digunakan ketika proses membutuhkan pengolahan node setelah seluruh cabang selesai diproses.

2.1.7 Method Traversal Inorder

```
47● void printInorder_1002(Node_2511531002 node_1002) {  
48    if (node_1002 == null)  
49        return;  
50    printInorder_1002(node_1002.left_1002);  
51    System.out.print(node_1002.data_1002 + " ");  
52    printInorder_1002(node_1002.right_1002);  
53 }
```

Kode Program 2.7 – Blok Program Method Traversal Inorder

Method ini digunakan untuk melakukan traversal tree dengan metode **Inorder**. Urutan kunjungannya adalah: **Subtree kiri – Root – Subtree kanan**. Pada struktur Binary Search Tree (BST), traversal inorder akan menghasilkan data dalam urutan menaik.

2.1.8 Method Menampilkan Struktur Tree

```
54● public String print_1002() {  
55    return this.print_1002("", true, "");  
56 }  
57● public String print_1002(String prefix_1002, boolean isTail_1002, String sb_1002) {  
58    if (right_1002 != null)  
59        right_1002.print_1002(prefix_1002 +  
60            (isTail_1002 ? "| " : " "), false, sb_1002);  
61    System.out.println(prefix_1002 + (isTail_1002 ?  
62        "\\-- " : "/-- ") + data_1002);  
63    if (left_1002 != null)  
64        left_1002.print_1002(prefix_1002 +  
65            (isTail_1002 ? " " : "| "), true, sb_1002);  
66    return sb_1002;  
67 }  
68 }
```

Kode Program 2.8 – Blok Program Method Print

Method **print_1002()** digunakan untuk menampilkan bentuk struktur tree secara visual pada console menggunakan karakter seperti \-- dan /--.

Method bekerja secara rekursif dengan mencetak node kanan terlebih dahulu, kemudian node saat ini, lalu node kiri. Parameter **prefix_1002** digunakan untuk mengatur jarak tampilan antar level, sedangkan **isTail_1002** menentukan bentuk garis cabang yang ditampilkan.

Hasil akhirnya membuat struktur tree lebih mudah diamati dibanding hanya menampilkan urutan traversal.

2.2 Implementasi Binary Tree

Class **BTree_2511531002** digunakan untuk mengelola keseluruhan struktur **binary tree**. Class ini bertanggung jawab dalam penyimpanan root, pencarian data, traversal, perhitungan jumlah node, serta pengaturan node aktif yang sedang digunakan.

2.2.1 Deklarasi Atribut

```
1 package pekan9_2511531002;  
2  
3 public class BTree_2511531002 {  
4     private Node_2511531002 root_1002;  
5     private Node_2511531002 currentNode_1002;
```

Kode Program 2.9– Blok Program Deklarasi Atribut

Bagian ini digunakan untuk menyimpan informasi utama dari tree. Variabel **root_1002** menyimpan node paling atas atau akar dari tree yang menjadi titik awal seluruh operasi. Variabel **currentNode_1002** digunakan untuk menyimpan node yang sedang aktif atau sedang diproses pada operasi tertentu.

2.2.2 Constructor BTree

```
6 public BTree_2511531002() {  
7     root_1002 = null;  
8 }
```

Kode Program 2.10 – Blok Program Constructor BTree

Constructor digunakan untuk membuat objek tree baru. Saat pertama kali dibuat, nilai **root_1002** diatur menjadi null yang menandakan bahwa tree masih kosong dan belum memiliki node.

2.2.3 Method Pencarian Data (Search)

```
9 public boolean search_1002(int data_1002) {  
10     return search_1002(root_1002, data_1002);  
11 }  
12 private boolean search_1002(Node_2511531002 node_1002, int data_1002) {  
13     if (node_1002.getData_1002() == data_1002)  
14         return true;  
15     if (node_1002.getLeft_1002() != null)  
16         if (search_1002(node_1002.getLeft_1002(), data_1002))  
17             return true;  
18     if (node_1002.getRight_1002() != null)  
19         if (search_1002(node_1002.getRight_1002(), data_1002))  
20             return true;  
21     return false;  
22 }
```

Kode Program 2.11 – Blok Program Pencarian Data

Method **search_1002()** digunakan untuk mencari apakah suatu nilai terdapat di dalam tree. Method publik menerima input berupa data yang dicari, kemudian proses pencarian dilakukan oleh method private menggunakan teknik **rekursif**. Alur pencariannya adalah sebagai berikut:

1. Membandingkan data node saat ini dengan data yang dicari.
2. Jika sama maka mengembalikan nilai true.
3. Jika belum ditemukan, pencarian dilanjutkan ke subtree kiri.
4. Jika masih belum ditemukan, pencarian dilanjutkan ke subtree kanan.
5. Jika seluruh node telah diperiksa dan tidak ditemukan, method mengembalikan false.

Pendekatan ini membuat proses pencarian dapat menjelajahi seluruh struktur tree hingga data ditemukan.

2.2.4 Method Traversal Tree

```
23 public void printInorder_1002() {  
24     root_1002.printInorder_1002(root_1002);  
25 }  
26 public void printPreOrder_1002() {  
27     root_1002.printPreorder_1002(root_1002);  
28 }  
29 public void printPostOrder_1002() {  
30     root_1002.printPostorder_1002(root_1002);  
31 }
```

Kode Program 2.12 – Blok Program Method Traversal Tree

Ketiga method ini digunakan untuk menjalankan proses traversal pada tree dengan memanfaatkan method traversal yang sebelumnya terdapat pada class **Node_2511531002**.

Fungsi masing-masing traversal:

- **printInorder_1002()** → menampilkan urutan **kiri – root – kanan**
- **printPreOrder_1002()** → menampilkan urutan **root – kiri – kanan**
- **printPostOrder_1002()** → menampilkan urutan **kiri – kanan – root**

Seluruh proses traversal dimulai dari node akar (root_1002).

2.2.5 Method Mengambil Root

```
33 public Node_2511531002 getRoot_1002() {  
34     return root_1002;  
35 }
```

Kode Program 2.13 – Blok Program Method Getter Root

Method ini digunakan untuk mengambil atau mengakses node akar dari tree. Nilai yang dikembalikan berupa objek **Node_2511531002** yang menjadi titik awal struktur tree.

2.2.6 Method Pemeriksaan Tree Kosong

```
36 public boolean isEmpty_1002() {  
37     return root_1002 == null;  
38 }
```

Kode Program 2.14 – Blok Program Method isEmpty

Method ini digunakan untuk memeriksa apakah tree masih kosong atau sudah memiliki data. Apabila **root_1002** bernilai null, method akan mengembalikan nilai **true**. Sebaliknya jika root sudah berisi node maka method mengembalikan **false**.

2.2.7 Method Menghitung Jumlah Node

```
39 public int countNodes_1002() {  
40     return countNodes_1002(root_1002);  
41 }  
42 private int countNodes_1002(Node_2511531002 node_1002) {  
43     int count = 1;  
44     if (node_1002 == null) {  
45         return 0;  
46     } else {  
47         count += countNodes_1002 (node_1002.getLeft_1002());  
48         count += countNodes_1002 (node_1002.getRight_1002());  
49         return count;  
50     }  
51 }
```

Kode Program 2.15 – Blok Program Method countNodes

Method ini digunakan untuk menghitung jumlah seluruh node yang terdapat pada tree. Perhitungan dilakukan secara rekursif dengan langkah:

1. Memulai perhitungan dari node saat ini.
2. Menghitung seluruh node pada subtree kiri.
3. Menghitung seluruh node pada subtree kanan.
4. Menjumlahkan seluruh hasil perhitungan.

Apabila node bernilai **null**, maka nilai yang dikembalikan adalah **0**.

2.2.8 Method Menampilkan Struktur Tree

```
53 public void print_1002() {  
54     root_1002.print_1002();  
55 }
```

Kode Program 2.16 – Blok Program Method print

Method ini digunakan untuk menampilkan bentuk struktur binary tree secara visual melalui console. Method akan memanggil fungsi **print_1002()** pada node akar sehingga seluruh cabang tree dapat ditampilkan sesuai susunan parent dan child yang terbentuk.

2.2.9 Method Getter dan Setter Current Node

```
56 public Node_2511531002 getCurrent_1002() {  
57     return currentNode_1002;  
58 }  
59  
60 public void setCurrent_1002(Node_2511531002 node_1002) {  
61     this.currentNode_1002 = node_1002;  
62 }  
63
```

Kode Program 2.17 – Blok Program Method getCurrent dan setCurrent

Method ini digunakan untuk mengambil maupun mengubah nilai **currentNode_1002**. **getCurrent_1002()** digunakan untuk memperoleh node aktif saat ini, sedangkan **setCurrent_1002()** digunakan untuk memperbarui node yang sedang diproses.

2.2.10 Method Setter Root

```
64 public void setRoot_1002(Node_2511531002 root) {  
65     this.root_1002 = root;  
66 }  
67 }
```

Kode Program 2.18 – Blok Program Method setRoot

Method **setRoot_1002()** digunakan untuk menetapkan atau mengganti node akar pada binary tree. Dengan adanya method ini, struktur tree dapat dibangun atau diperbarui dari node tertentu yang ditentukan sebagai root.

2.3 Implementasi Binary Tree Driver

Class **BtreeDriver_2511531002** digunakan sebagai program utama (*driver class*) untuk menjalankan simulasi pembuatan dan pengujian struktur **binary tree**. Pada class ini dilakukan proses pembuatan objek tree, penambahan node, pengaturan root, traversal, serta menampilkan bentuk akhir tree.

2.3.1 Method membuat Tree

```
1 package pekan9_2511531002;
2
3 public class BtreeDriver_2511531002 {
4     public static void main(String[] args) {
5         // Membuat Pohon
6         BTree_2511531002 tree_1002 = new BTree_2511531002();
7         System.out.print("Jumlah Simpul awal pohon: ");
8         System.out.println(tree_1002.countNodes_1002());
```

Kode Program 2.19 – Blok Program Membuat objek Tree

Bagian ini digunakan untuk membuat objek baru dari class **BTree_2511531002** yang akan digunakan sebagai struktur penyimpanan data berbentuk binary tree.

Setelah objek dibuat, program langsung menampilkan jumlah simpul yang terdapat pada tree menggunakan method **countNodes_1002()**. Karena tree masih kosong dan belum memiliki root, jumlah simpul yang ditampilkan adalah **0**.

2.3.2 Method Menambahkan Simpul Data

```
10         // Menambahkan Simpul Data
11         Node_2511531002 root_1002 = new Node_2511531002(1);
```

Kode Program 2.20 – Blok Program Menambahkan Simpul Data

Bagian ini digunakan untuk membuat simpul pertama dengan nilai **1**. Node tersebut belum langsung menjadi bagian dari tree, tetapi hanya dibuat sebagai objek yang nantinya akan dijadikan akar (*root*) dari struktur binary tree.

2.3.3 Menjadikan simpul 1 sebagai Root

```
13      // Menjadikan Simpul 1 Sebagai Root
14      tree_1002.setRoot_1002(root_1002);
15      System.out.println("Jumlah simpul jika hanya ada root");
16      System.out.println(tree_1002.countNodes_1002());
17      Node_2511531002 node2_1002 = new Node_2511531002(2);
18      Node_2511531002 node3_1002 = new Node_2511531002(3);
19      Node_2511531002 node4_1002 = new Node_2511531002(4);
20      Node_2511531002 node5_1002 = new Node_2511531002(5);
21      Node_2511531002 node6_1002 = new Node_2511531002(6);
22      Node_2511531002 node7_1002 = new Node_2511531002(7);
23      Node_2511531002 node8_1002 = new Node_2511531002(8);
24      Node_2511531002 node9_1002 = new Node_2511531002(9);
25      root_1002.setLeft_1002(node2_1002);
26      node2_1002.setLeft_1002(node4_1002);
27      node2_1002.setRight_1002(node5_1002);
28      node4_1002.setRight_1002(node8_1002);
29      root_1002.setRight_1002(node3_1002);
30      node3_1002.setLeft_1002(node6_1002);
31      node3_1002.setRight_1002(node7_1002);
32      node6_1002.setLeft_1002(node9_1002);
```

Kode Program 2.21 – Blok Program menentukan root

Bagian ini digunakan untuk menetapkan node bernilai **1** sebagai akar dari tree menggunakan method **setRoot_1002()**. Setelah root ditentukan, dilakukan pengecekan jumlah simpul menggunakan **countNodes_1002()**. Karena baru terdapat satu node pada tree, hasil yang ditampilkan adalah **1**.

Selanjutnya dibuat node-node tambahan yang akan digunakan untuk membentuk cabang pada tree. Kemudian setiap node dihubungkan menggunakan method **setLeft_1002()** dan **setRight_1002()**.

2.3.4 Method Set root

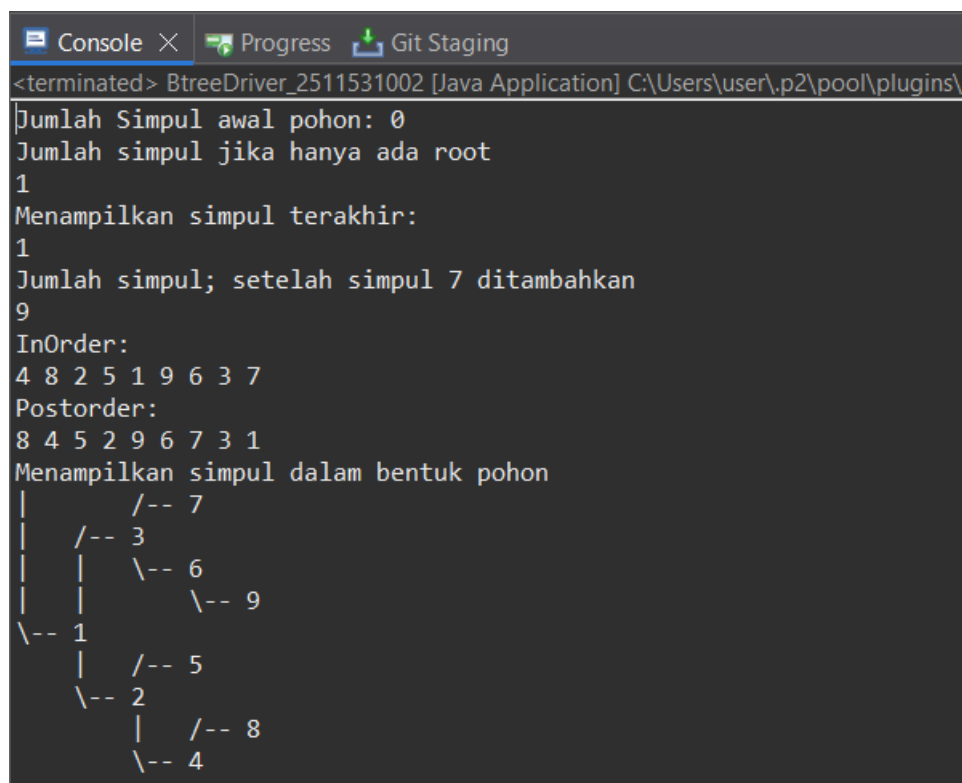
```
34      // Set Root
35      tree_1002.setCurrent_1002(tree_1002.getRoot_1002());
36      System.out.println("Menampilkan simpul terakhir: ");
37      System.out.println(tree_1002.getCurrent_1002().getData_1002());
38      System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
39      System.out.println(tree_1002.countNodes_1002());
40      System.out.println("InOrder: ");
41      tree_1002.printInorder_1002();
42      System.out.println("\nPostorder: ");
43      tree_1002.printPostOrder_1002();
44      System.out.println("\nMenampilkan simpul dalam bentuk pohon");
45      tree_1002.print_1002();
46  }
47 }
```

Kode Program 2.22 – Blok Program Set Root

Bagian ini digunakan untuk menentukan node aktif (*current node*) dengan mengambil node root menggunakan **getRoot_1002()** lalu menyimpannya melalui **setCurrent_1002()**. Setelah itu program melakukan beberapa pengujian terhadap tree yang telah dibuat, antara lain:

- Menampilkan data dari node aktif menggunakan **getCurrent_1002()**
- Menghitung jumlah seluruh simpul menggunakan **countNodes_1002()**
- Menampilkan traversal **Inorder**
- Menampilkan traversal **Postorder**
- Menampilkan visualisasi struktur tree menggunakan **print_1002()**

Tahap ini dilakukan untuk memastikan bahwa seluruh node sudah terhubung dengan benar dan tree dapat dijalankan sesuai rancangan. Hasil output dari program ini adalah sebagai berikut:



```
<terminated> BtreeDriver_2511531002 [Java Application] C:\Users\user\.p2\pool\plugins\
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|      /-- 7
|      /-- 3
|      |      \-- 6
|      |      \-- 9
|      |
|-- 1
|      /-- 5
|-- 2
|      |      /-- 8
|      |      \-- 4
```

Gambar 2. 1 – Hasil Program Binary Tree Driver

2.4 Implementasi Graph Traversal

Class **GraphTraversal_2511531002** digunakan untuk merepresentasikan struktur **graf tak berarah (undirected graph)** menggunakan **Adjacency List**. Class ini menyediakan fungsi untuk menambahkan hubungan antar simpul, menampilkan graf, serta melakukan penelusuran menggunakan algoritma **Depth First Search (DFS)** dan **Breadth First Search (BFS)**.

2.4.1 Deklarasi Atribut Graph

```
1 package pekan9_2511531002;
2
3 import java.util.*;
4
5 public class GraphTraversal_2511531002 {
6     private Map<String, List<String>> graph_1002 = new HashMap<>();
```

Kode Program 2.23 – Blok Program Deklarasi Atribut Graph

Bagian ini digunakan untuk menyimpan struktur graf menggunakan **Adjacency List**. Map digunakan untuk menyimpan pasangan antara nama simpul dan daftar tetangganya. Setiap key merepresentasikan simpul, sedangkan value berupa List yang menyimpan node-node yang terhubung. Pendekatan adjacency list dipilih karena lebih efisien dalam menyimpan hubungan antar simpul dibanding matriks jika jumlah sisi tidak terlalu banyak.

2.4.2 Menambahkan Edge pada Graf

```
8 // Menambahkan edge (graf tak berarah)
9 public void addEdge_1002(String node1_1002, String node2_1002) {
10     graph_1002.putIfAbsent(node1_1002, new ArrayList<>());
11     graph_1002.putIfAbsent(node2_1002, new ArrayList<>());
12     graph_1002.get(node1_1002).add(node2_1002);
13     graph_1002.get(node2_1002).add(node1_1002);
14 }
```

Kode Program 2.24 – Blok Program method addEdge

Method **addEdge_1002()** digunakan untuk menambahkan hubungan antar dua simpul pada graf. Langkah kerja method adalah sebagai berikut:

1. Memastikan kedua simpul sudah tersedia di dalam Map.
2. Jika belum tersedia maka dibuat list baru.

3. Menambahkan node kedua ke daftar tetangga node pertama.
4. Menambahkan node pertama ke daftar tetangga node kedua.

Karena hubungan dilakukan dua arah, graf yang dihasilkan termasuk **graf tak berarah (undirected graph)**.

2.4.3 Menampilkan Graph Awal

```

16 // Menambahkan graf awal
17 public void printGraph_1002() {
18     System.out.println("Graf Awal (Adjacency List): ");
19     for (String node_1002 : graph_1002.keySet()) {
20         System.out.print(node_1002 + " → ");
21         List<String> neighbors_1002 = graph_1002.get(node_1002);
22         System.out.println(String.join(" ", neighbors_1002));
23     }
24     System.out.println();
25 }

```

Kode Program 2.25 – Blok Program method printGraph

Method ini digunakan untuk menampilkan isi graf dalam bentuk **Adjacency List**. Program melakukan perulangan pada seluruh key di dalam Map, kemudian mengambil daftar tetangga dari masing-masing simpul dan menampilkannya ke layar.

2.4.4 Traversal menggunakan DFS (Depth-First Search)

```

27 // DFS Rekursif
28 public void dfs_1002(String start_1002) {
29     Set<String> visited_1002 = new HashSet<>();
30     System.out.println("Penelusuran DFS: ");
31     dfsHelper_1002(start_1002, visited_1002);
32     System.out.println();
33 }
34 public void dfsHelper_1002(String current_1002, Set<String> visited_1002) {
35     if (visited_1002.contains(current_1002)) return;
36     visited_1002.add(current_1002);
37     System.out.print(current_1002 + " ");
38     for (String neighbor_1002 : graph_1002.getOrDefault(current_1002, new ArrayList<>()))
39         dfsHelper_1002(neighbor_1002, visited_1002);
40 }

```

Kode Program 2.26 – Blok Program Method DFS

Method ini digunakan untuk melakukan penelusuran graf menggunakan algoritma **Depth First Search (DFS)**. Proses dimulai dari simpul awal yang diberikan, kemudian:

1. Simpul ditandai sebagai telah dikunjungi.
2. Simpul dicetak ke layar.

3. Program berpindah ke tetangga pertama.
4. Penelusuran dilakukan terus secara rekursif hingga tidak ada jalur baru.
5. Setelah selesai, program kembali ke simpul sebelumnya untuk mengecek cabang lain.

Struktur **Set** digunakan agar simpul yang sudah dikunjungi tidak diproses kembali.

2.4.5 Traversal menggunakan BFS (Breadth-First Search)

```

42 // BFS Iteratif
43 public void bfs_1002(String start_1002) {
44     Set<String> visited_1002 = new HashSet<>();
45     Queue<String> queue_1002 = new LinkedList<>();
46     queue_1002.add(start_1002);
47     visited_1002.add(start_1002);
48     System.out.println("Penelusuran DFS: ");
49     while (!queue_1002.isEmpty()) {
50         String current_1002 = queue_1002.poll();
51         System.out.print(current_1002 + " ");
52         for (String neighbor_1002 : graph_1002.getOrDefault(current_1002, new ArrayList<>())) {
53             if (!visited_1002.contains(neighbor_1002)) {
54                 queue_1002.add(neighbor_1002);
55                 visited_1002.add(neighbor_1002);
56             }
57         }
58     }
59     System.out.println();
60 }

```

Kode Program 2.27 – Blok Program Method BFS

Method ini digunakan untuk melakukan penelusuran menggunakan algoritma **Breadth First Search (BFS)**. Berbeda dengan DFS, BFS menelusuri simpul berdasarkan urutan tingkat (*level*) menggunakan struktur data **Queue**. Langkah kerja:

1. Simpul awal dimasukkan ke queue.
2. Simpul diambil dari depan queue.
3. Simpul dicetak dan ditandai telah dikunjungi.
4. Semua tetangga yang belum dikunjungi dimasukkan ke queue.
5. Proses diulang hingga queue kosong.

Pendekatan ini menghasilkan penelusuran yang menyebar ke seluruh tetangga sebelum berpindah ke tingkat berikutnya.

2.4.6 Implementasi dan Pengujian Graf

```
62 // Main
63 public static void main(String[] args) {
64     GraphTraversal_2511531002 graph_1002 = new GraphTraversal_2511531002();
65
66     // Contoh graf: A-B, A-C, B-D, B-E
67     graph_1002.addEdge_1002("A", "B");
68     graph_1002.addEdge_1002("A", "C");
69     graph_1002.addEdge_1002("B", "D");
70     graph_1002.addEdge_1002("B", "E");
71
72     // Cetak graf awal
73     System.out.println("Graf Awal adalah: ");
74     graph_1002.printGraph_1002();
75
76     // Lakukan penelusuran
77     graph_1002.dfs_1002("A");
78     graph_1002.bfs_1002("A");
79 }
80 }
```

Kode Program 2.28 – Blok Program Main

Bagian ini digunakan untuk melakukan pengujian terhadap seluruh fungsi yang terdapat pada class **GraphTraversal_2511531002**.

Program diawali dengan membuat objek **graph_1002** yang akan digunakan sebagai wadah penyimpanan struktur graf. Setelah objek dibuat, dilakukan penambahan hubungan antar simpul menggunakan method **addEdge_1002()** sehingga terbentuk hubungan **A-B**, **A-C**, **B-D**, dan **B-E**.

Setelah seluruh hubungan ditambahkan, program menampilkan graf awal menggunakan method **printGraph_1002()** dalam bentuk **Adjacency List** untuk menunjukkan keterhubungan antar simpul.

Tahap berikutnya adalah melakukan proses traversal menggunakan algoritma **Depth First Search (DFS)** dimulai dari simpul **A**, kemudian dilanjutkan menggunakan algoritma **Breadth First Search (BFS)** dari titik awal yang sama.

Pengujian ini dilakukan untuk memastikan bahwa struktur graf telah terbentuk dengan benar serta membandingkan hasil penelusuran yang dihasilkan oleh kedua metode traversal. Hasil output dari program adalah sebagai berikut:

```
Console X Progress Git Staging
<terminated> GraphTraversal_2511531002 [Java]
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran DFS:
A B C D E
```

Gambar 2. 2 – Hasil Program Graph Traversal

2.5 Latihan Membuat Peta Bandara dan Pencarian Interaktif menggunakan BFS dan DFS dengan menggunakan GUI

```
package pekan9_2511531002;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.*;
import java.util.List;
import java.util.*;
import javax.swing.*;

public class PetaBandara_2511531002 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1002;
    private GraphPanel_1002 pnlGraph_1002;
    private JComboBox<String> cbStart_1002;
    private JComboBox<String> cbGoal_1002;
    private JTextArea txtHasil_1002;
    private HashMap<String, ArrayList<String>>
graph_1002;
    private HashMap<String, Point> positions_1002;
    private Set<String> visitedNodes_1002 = new
HashSet<>();
    private List<String> path_1002 = new
ArrayList<>();

    public PetaBandara_2511531002() {
```

```

        setTitle("Peta Bandara BFS DFS -
2511531002");

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1100,700);
        setLocationRelativeTo(null);

        contentPane_1002 = new JPanel();
        contentPane_1002.setLayout(null);

        setContentPane(contentPane_1002);

        pnlGraph_1002 = new GraphPanel_1002();
        pnlGraph_1002.setBounds(10,10,740,620);
        contentPane_1002.add(pnlGraph_1002);

        JLabel lblStart_1002 = new JLabel("Start");
        lblStart_1002.setBounds(770,20,100,25);
        contentPane_1002.add(lblStart_1002);

        cbStart_1002 = new JComboBox<>();
        cbStart_1002.setBounds(770,45,250,30);
        contentPane_1002.add(cbStart_1002);

        JLabel lblGoal_1002 = new JLabel("Goal");
        lblGoal_1002.setBounds(770,90,100,25);
        contentPane_1002.add(lblGoal_1002);

        cbGoal_1002 = new JComboBox<>();
        cbGoal_1002.setBounds(770, 115, 250, 30);
        contentPane_1002.add(cbGoal_1002);

        JButton btnBFS_1002 = new JButton("BFS");
        btnBFS_1002.setBounds(770,170,80,35);
        contentPane_1002.add(btnBFS_1002);

        JButton btnDFS_1002 = new JButton("DFS");
        btnDFS_1002.setBounds(860,170,80,35);
        contentPane_1002.add(btnDFS_1002);

        JButton btnReset_1002 = new
JButton("Reset");
        btnReset_1002.setBounds(950,170,80,35);
        contentPane_1002.add(btnReset_1002);

        txtHasil_1002 = new JTextArea();
        txtHasil_1002.setEditable(false);

```

```

        JScrollPane scroll_1002 = new
JScrollPane(txtHasil_1002);
        scroll_1002.setBounds(770,230,280,380);
        contentPane_1002.add(scroll_1002);

        initGraph_1002();
        initPositions_1002();

        btnBFS_1002.addActionListener(e ->
        BFS_1002(

        cbStart_1002.getSelectedItem().toString(),

        cbGoal_1002.getSelectedItem().toString()
        )
        );

        btnDFS_1002.addActionListener(e ->
        DFS_1002(

        cbStart_1002.getSelectedItem().toString(),

        cbGoal_1002.getSelectedItem().toString()
        )
        );

        btnReset_1002.addActionListener(e ->
        resetGraph_1002());
    }

    private void initGraph_1002() {
        graph_1002 = new HashMap<>();
        String[] nodes_1002 = {
            "Parking",
            "DropOffZone",
            "CheckInCounter",
            "SecurityCheck",
            "Immigration",
            "DutyFreeShop",
            "AirportLounge",
            "TerminalA",
            "TerminalB",
            "SkyTrain",
            "BaggageClaim",
            "ArrivalExit"
        };

        for(String node_1002 : nodes_1002) {

```

```

graph_1002.put(node_1002, new
ArrayList<>());
cbStart_1002.addItem(node_1002);
cbGoal_1002.addItem(node_1002);
    }

    addEdge_1002("Parking", "DropOffZone");
    addEdge_1002("Parking", "CheckInCounter");
    addEdge_1002("DropOffZone", "CheckInCounter");

addEdge_1002("CheckInCounter", "SecurityCheck");

addEdge_1002("CheckInCounter", "DutyFreeShop");
    addEdge_1002("SecurityCheck", "Immigration");

addEdge_1002("SecurityCheck", "AirportLounge");
    addEdge_1002("Immigration", "DutyFreeShop");
    addEdge_1002("DutyFreeShop", "AirportLounge");
    addEdge_1002("DutyFreeShop", "TerminalA");
    addEdge_1002("DutyFreeShop", "TerminalB");
    addEdge_1002("AirportLounge", "TerminalA");
    addEdge_1002("AirportLounge", "TerminalB");
    addEdge_1002("TerminalA", "TerminalB");
    addEdge_1002("TerminalA", "SkyTrain");
    addEdge_1002("TerminalB", "SkyTrain");
    addEdge_1002("SkyTrain", "BaggageClaim");
    addEdge_1002("BaggageClaim", "ArrivalExit");
    }

    private void addEdge_1002(String from_1002,
String to_1002) {
        graph_1002.get(from_1002).add(to_1002);
        graph_1002.get(to_1002).add(from_1002);
    }

    private void initPositions_1002() {
        positions_1002 = new HashMap<>();

        positions_1002.put("Parking", new
Point(120, 500));
        positions_1002.put("DropOffZone", new
Point(280, 500));
        positions_1002.put("CheckInCounter", new
Point(200, 400));
        positions_1002.put("SecurityCheck", new
Point(200, 300));
        positions_1002.put("Immigration", new
Point(100, 200));
    }

```

```

        positions_1002.put("DutyFreeShop", new
Point(300,200));
        positions_1002.put("AirportLounge", new
Point(300,100));
        positions_1002.put("TerminalA", new
Point(480,80));
        positions_1002.put("TerminalB", new
Point(480,200));
        positions_1002.put("SkyTrain", new
Point(620,140));
        positions_1002.put("BaggageClaim", new
Point(620,300));
        positions_1002.put("ArrivalExit", new
Point(620,450));
    }

    class GraphPanel_1002 extends JPanel {
        public GraphPanel_1002() {
setBackground(Color.WHITE);}

        @Override
        protected void paintComponent(Graphics
g_1002) {
            super.paintComponent(g_1002);
            Graphics2D g2_1002 = (Graphics2D)
g_1002;

            for (String node_1002 :
graph_1002.keySet()) {
                Point p1_1002 =
positions_1002.get(node_1002);
                for (String next_1002 :
graph_1002.get(node_1002)) {
                    Point p2_1002 =
positions_1002.get(next_1002);

                    g2_1002.setColor(Color.BLACK);
                    g2_1002.drawLine(p1_1002.x,
p1_1002.y, p2_1002.x, p2_1002.y);
                }
            }

            for (String node_1002 :
graph_1002.keySet()) {
                Point p_1002 =
positions_1002.get(node_1002);

                if(path_1002.contains(node_1002))

```

```

        g2_1002.setColor(Color.GREEN);
        else
    if(visitedNodes_1002.contains(node_1002))

        g2_1002.setColor(Color.YELLOW);
        else

        g2_1002.setColor(Color.LIGHT_GRAY);

        g2_1002.fillOval(p_1002.x-25,
p_1002.y-25, 50, 50);
        g2_1002.setColor(Color.BLACK);
        g2_1002.drawOval(p_1002.x-25,
p_1002.y-25, 50, 50);
        g2_1002.drawString(node_1002,
p_1002.x-35, p_1002.y+40);
    }
    }

    private void BFS_1002(String start_1002, String
goal_1002) {
        Queue<String> q_1002 = new LinkedList<>();
        HashSet<String> visited_1002 = new
HashSet<>();
        HashMap<String,String> parent_1002 = new
HashMap<>();
        ArrayList<String> order_1002 = new
ArrayList<>();

        q_1002.add(start_1002);
        visited_1002.add(start_1002);

        while(!q_1002.isEmpty()) {
            String curr_1002 = q_1002.poll();
            order_1002.add(curr_1002);
            if (curr_1002.equals(goal_1002))
                break;

            for(String n_1002 :
graph_1002.get(curr_1002)) {
                if(!visited_1002.contains(n_1002))
                {
                    visited_1002.add(n_1002);
                    parent_1002.put(n_1002,
curr_1002);
                    q_1002.add(n_1002);
                }
            }
        }
    }
}

```

```

    }
    }
    }

    visitedNodes_1002 = visited_1002;
    path_1002 = buildPath_1002(
        parent_1002, start_1002,
goal_1002);

    txtHasil_1002.setText(
        "ALGORITMA : BFS\n\n" +
        "Path : " + path_1002 +
        "\n\nVisited : " + order_1002 +
        "\n\nJumlah Node Dieksplorasi : "
        + order_1002.size());
    pnlGraph_1002.repaint();
}

private void DFS_1002(String start_1002, String
goal_1002) {
    Stack<String> stack_1002 = new Stack<>();
    HashSet<String> visited_1002 = new
HashSet<>();
    HashMap<String, String> parent_1002 = new
HashMap<>();
    ArrayList<String> order_1002 = new
ArrayList<>();

    stack_1002.push(start_1002);
    while(!stack_1002.isEmpty()) {
        String curr_1002 = stack_1002.pop();

        if(visited_1002.contains(curr_1002))
            continue;

        visited_1002.add(curr_1002);
        order_1002.add(curr_1002);

        if(curr_1002.equals(goal_1002))
            break;

        List<String> neighbors_1002 =
graph_1002.get(curr_1002);
        for(int i_1002 = neighbors_1002.size()-
1; i_1002 >= 0; i_1002--) {
            String next_1002 =
neighbors_1002.get(i_1002);

```

```

        if(!visited_1002.contains(next_1002)) {
            if(!parent_1002.containsKey(next_1002))
                parent_1002.put(next_1002, curr_1002);
            stack_1002.push(next_1002);
        }
    }

    visitedNodes_1002 = visited_1002;
    path_1002 = buildPath_1002(parent_1002,
start_1002, goal_1002);

    txtHasil_1002.setText(
        "ALGORITMA : DFS\n\n" +
        "Path : " + path_1002 +
        "\n\nVisited : " + order_1002 +
        "\n\nJumlah Node Dieksplorasi : "
        + order_1002.size());
    pnlGraph_1002.repaint();
}

private ArrayList<String> buildPath_1002(
    HashMap<String,String> parent_1002,
    String start_1002, String goal_1002){

    ArrayList<String> result_1002 = new
ArrayList<>();
    if(!start_1002.equals(goal_1002)
        &&
!parent_1002.containsKey(goal_1002))
        return result_1002;

    String curr_1002 = goal_1002;
    while(curr_1002 != null) {
        result_1002.add(0, curr_1002);
        curr_1002 = parent_1002.get(curr_1002);
    }

    return result_1002;
}

private void resetGraph_1002() {
    visitedNodes_1002.clear();
    path_1002.clear();
    txtHasil_1002.setText("");
}

```

```

        pnlGraph_1002.repaint();
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    PetaBandara_2511531002 frame
= new PetaBandara_2511531002();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }
}

```

Kode Program 2.29 – Tugas Latihan Praktikum

Class **PetaBandara_2511531002** merupakan implementasi visualisasi pencarian jalur menggunakan algoritma **Breadth First Search (BFS)** dan **Depth First Search (DFS)** pada lingkungan bandara berbasis **Java Swing GUI**. Program ini dibuat untuk mensimulasikan perpindahan antar lokasi di dalam area bandara serta membandingkan cara kerja kedua algoritma traversal dalam menemukan jalur dari lokasi awal menuju lokasi tujuan.

Program dibangun menggunakan komponen antarmuka berupa **JFrame**, **JPanel**, **JComboBox**, **JButton**, dan **JTextArea**. Area utama digunakan untuk menampilkan graf bandara dalam bentuk visual, sedangkan panel kontrol digunakan untuk memilih lokasi awal (*start*), lokasi tujuan (*goal*), menjalankan algoritma BFS atau DFS, serta melakukan reset hasil pencarian.

Struktur bandara direpresentasikan menggunakan **graf tak berarah (undirected graph)** yang disimpan dalam **HashMap<String, ArrayList<String>>**, di mana setiap lokasi menyimpan daftar lokasi lain yang terhubung secara langsung. Node yang digunakan meliputi area seperti **Parking**, **DropOffZone**, **CheckInCounter**, **SecurityCheck**, **Immigration**, **DutyFreeShop**, **AirportLounge**, **TerminalA**, **TerminalB**, **SkyTrain**, **BaggageClaim**, hingga

ArrivalExit. Selain hubungan antar node, setiap lokasi juga diberikan koordinat menggunakan objek Point agar dapat divisualisasikan pada panel graf.

Ketika program dijalankan, sistem terlebih dahulu melakukan inisialisasi graf beserta posisi masing-masing node. Data lokasi kemudian dimasukkan ke dalam ComboBox sehingga pengguna dapat memilih titik awal dan tujuan pencarian. Setelah pengguna menekan tombol **BFS** atau **DFS**, sistem akan menjalankan algoritma yang dipilih untuk menelusuri graf.

Pada algoritma **BFS**, proses pencarian dilakukan menggunakan struktur data **Queue** sehingga penelusuran berlangsung berdasarkan tingkat kedalaman yang sama terlebih dahulu (*level order traversal*). Setiap node yang dikunjungi disimpan ke dalam daftar visited, sedangkan hubungan antar node dicatat menggunakan struktur parent untuk membangun kembali jalur akhir menuju tujuan.

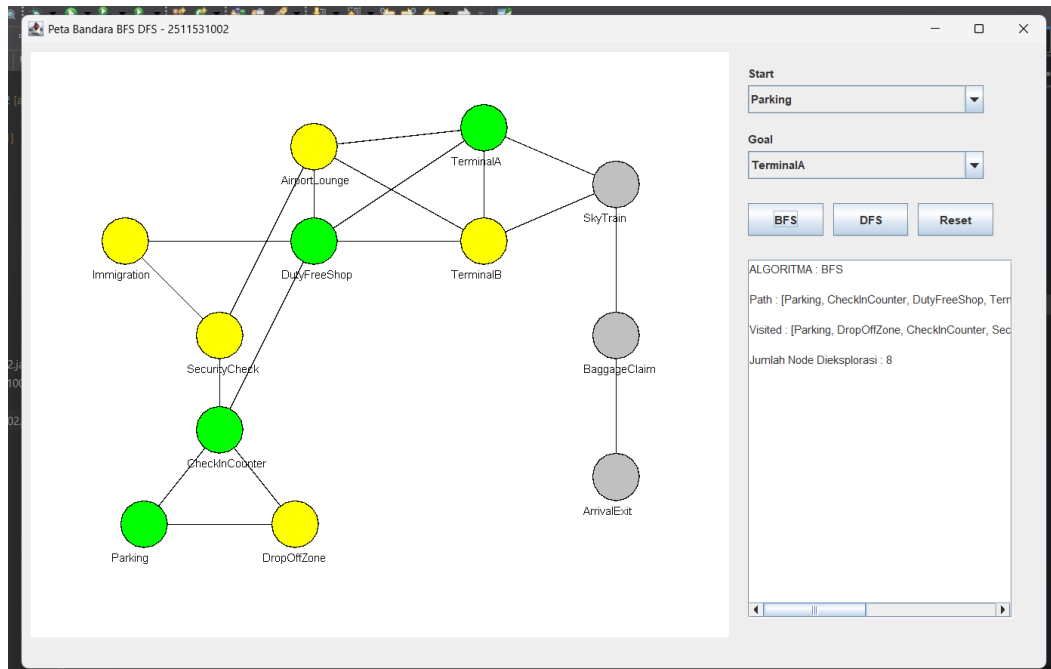
Sementara itu pada algoritma **DFS**, pencarian dilakukan menggunakan **Stack** sehingga sistem akan menelusuri satu jalur sedalam mungkin sebelum kembali ke cabang sebelumnya. Sama seperti BFS, node yang dikunjungi dan hubungan parent tetap dicatat agar jalur hasil pencarian dapat direkonstruksi.

Setelah proses traversal selesai, sistem membangun kembali rute menggunakan method **buildPath_1002()** yang menelusuri parent dari node tujuan hingga kembali ke titik awal. Hasil pencarian kemudian ditampilkan pada area output berupa nama algoritma yang digunakan, jalur yang ditemukan (*path*), urutan node yang dikunjungi (*visited*), serta jumlah node yang dieksplorasi selama proses pencarian.

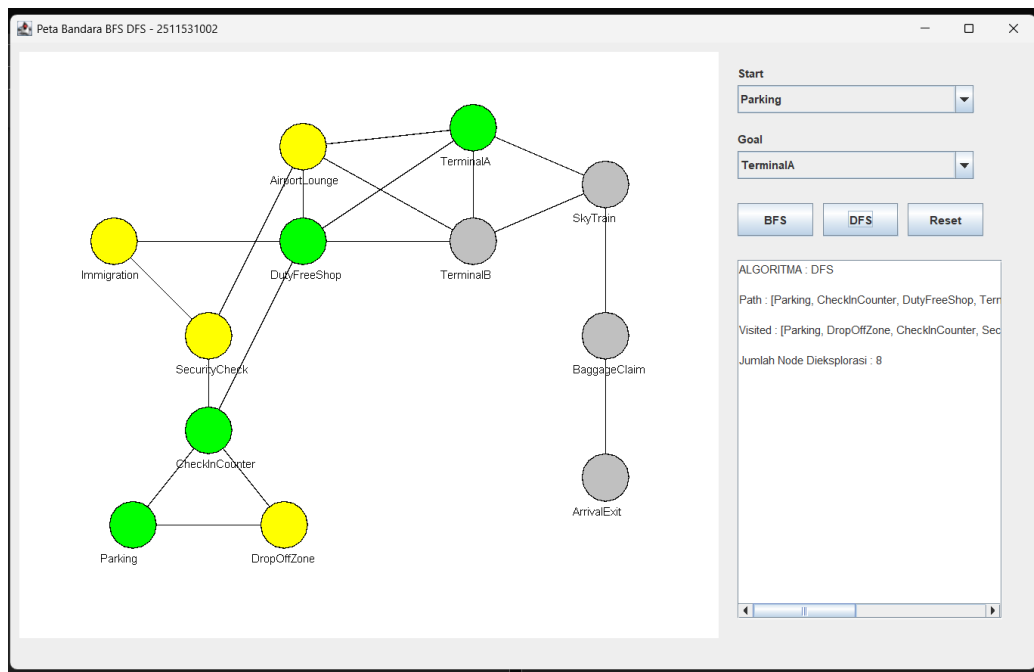
Visualisasi hasil pencarian ditampilkan melalui class **GraphPanel_1002**. Node yang termasuk jalur akhir ditampilkan menggunakan warna **hijau**, node yang telah dikunjungi tetapi bukan bagian jalur ditampilkan menggunakan warna **kuning**, sedangkan node yang belum diproses ditampilkan dengan warna **abu-abu**. Selain itu, tombol **Reset** disediakan untuk menghapus hasil traversal sebelumnya sehingga pengguna dapat melakukan pengujian ulang menggunakan kombinasi lokasi dan algoritma yang berbeda.

Melalui implementasi ini, pengguna dapat mengamati secara langsung perbedaan pola pencarian antara algoritma BFS dan DFS baik dari urutan eksplorasi maupun jalur yang dihasilkan pada struktur graf yang sama.

Hasil output program akan terlihat seperti berikut:



Gambar 2. 3 – Hasil Program PetaBandara_2511531002 dengan Metode BFS



Gambar 2. 4 – Hasil Program PetaBandara_2511531002 dengan Metode DFS

BAB III

KESIMPULAN

3.1 Kesimpulan

berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa struktur data Binary Tree dan Graph dapat digunakan untuk merepresentasikan hubungan antar data dengan pendekatan yang berbeda sesuai kebutuhan.

Pada implementasi Binary Tree, proses traversal menggunakan metode preorder, inorder, dan postorder menunjukkan bahwa urutan kunjungan node memengaruhi hasil penelusuran yang diperoleh. Selain itu, proses pencarian dan perhitungan jumlah simpul dapat dilakukan menggunakan pendekatan rekursif.

Pada implementasi Graph, algoritma Breadth First Search (BFS) dan Depth First Search (DFS) berhasil digunakan untuk melakukan pencarian jalur dari lokasi awal menuju lokasi tujuan. BFS melakukan penelusuran berdasarkan tingkat sehingga cenderung menemukan jalur dengan jumlah langkah lebih sedikit, sedangkan DFS menelusuri satu cabang hingga dalam sebelum berpindah ke cabang lainnya.

Visualisasi menggunakan Java Swing membantu memperlihatkan proses traversal secara lebih jelas melalui perubahan warna node, jalur yang ditemukan, serta informasi urutan eksplorasi. Dengan demikian, praktikum ini memberikan pemahaman mengenai penerapan struktur data dan algoritma traversal dalam bentuk implementasi program yang dapat diamati secara langsung.

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 9